

A portfolio optimization model using Genetic Network Programming with control nodes

Yan Chen ^{a,*}, Etsushi Ohkawa ^a, Shingo Mabu ^a, Kaoru Shimada ^b, Kotaro Hirasawa ^{a,*}

^a Graduate school of Information, Production and Systems, Waseda University, 2-7, Hibikino, Wakamatsu-ku, Kitakyushu, Fukuoka 808-0135, Japan

^b Information, Production and Systems Research Center, Waseda University, 2-7, Hibikino, Wakamatsu-ku, Kitakyushu, Fukuoka, Japan

ARTICLE INFO

Keywords:

Portfolio optimization
Genetic Network Programming
Control node
Reinforcement learning

ABSTRACT

Many evolutionary computation methods applied to the financial field have been reported. A new evolutionary method named “Genetic Network Programming” (GNP) has been developed and applied to the stock market recently. The efficient trading rules created by GNP has been confirmed in our previous research. In this paper a multi-brands portfolio optimization model based on Genetic Network Programming with control nodes is presented. This method makes use of the information from technical indices and candlestick chart. The proposed optimization model, consisting of technical analysis rules, are trained to generate trading advice. The experimental results on the Japanese stock market show that the proposed optimization system using GNP with control nodes method outperforms other traditional models in terms of both accuracy and efficiency. We also compared the experimental results of the proposed model with the conventional GNP based methods, GA and Buy&Hold method to confirm its effectiveness, and it is clarified that the proposed trading model can obtain much higher profits than these methods.

© 2009 Elsevier Ltd. All rights reserved.

1. Introduction

Nowadays, evolutionary computation has become a subject of general interest with regard to the power to solve complex optimization problems. It has been successfully applied to many fields of science and technology. This paper presents an application of evolutionary computation method named Genetic Network Programming to the problem of multi-brands optimization in the field of financial economics, which is one kind of portfolio optimization. Portfolio optimization in the stock market consists of deciding what brands to include in a portfolio given the investor's objectives and economic conditions. The always difficult selection process includes identifying which brands to purchase, how much, and when. The basic idea is that we want to choose a group of brands from a large number of available issues, in order to maximize the expected return given an acceptable risk rate. A rational investor needs to consider not only maximizing the profit of the investment, but also minimizing the uncertainty or risk resulting from the fluctuations that are expected in the value of the portfolio. The main problem is how to allocate the available capital in order to maximize profit and minimize risk simultaneously.

There have been increased the number of applications of Artificial Intelligence (AI) techniques, mainly artificial neural networks,

genetic algorithm and genetic programming, which have been applied to technical financial forecasting (Dempster & Jones, 2001; Goldberg, 1989) as they have the ability to deal with complex non-linear problems and have the self-adaptation for dynamically changing problems. Several applications of Genetic Algorithms (GA) to the financial problems have been done, such as portfolio optimization, bankruptcy prediction, financial forecasting, fraud detection and scheduling (Loraschi, Tettamanzi, Tomassini, & Verda, 1995; Skolpadungket, Dahal, & Harnpornchai, 2007). Genetic Programming (GP) (Koza, 1992) has also been applied to many problems in the time-series prediction. Although these AI approaches possess the properties required for the technical financial forecasting, they have some inherent bottlenecks. For example, Genetic Programming sometimes causes the bloating problem due to its tree structure. Neural networks cannot be used to explain the causal relationships between input and output variables because of their black-box nature.

In the past studies, we have proposed Genetic Network Programming (GNP) and Genetic Network Programming with Reinforcement Learning (GNP-RL) (Eguchi, Hirasawa, Hu, & Ota, 2006; Mabu, Chen, Hirasawa, & Hu, 2007; Mabu, Hirasawa, & Hu, 2007) as an extended method of GA (Goldberg, 1989) and GP (Koza, 1992). Since GNP represents its solutions using graph structures, which contributes to creating quite compact programs and implicitly memorizing past action sequences in the network flows, it has been clarified that GNP is an effective method mainly for complicated problems such as portfolio optimization systems. Moreover,

* Corresponding author.

E-mail addresses: chenyan15153@akane.waseda.jp (Y. Chen), hirasawa@waseda.jp (K. Hirasawa).

in our former research, GNP-RL method was successfully applied to stock trading model (Mabu et al., 2007), and its applicability and efficiency has been confirmed.

Recently, in order to extend the functions of conventional GNP, Genetic Network Programming with control nodes (GNPcn) (Eto, Mabu, Hirasawa, & Hu, 2006) has been proposed. Since GNP has a directed graph structure, the aim of GNPcn is to improve the performance of GNP by extending the evolutionary method of it. In traditional GNP, the current node isn't compulsorily transferred to the start node. However, in the GNPcn method, the number of control nodes are set up and a certain number of processing nodes are executed before returning to one of the control nodes, i.e., we extend the breadth and depth of searching space for GNP. It is clarified from the simulations that the performance of GNP could be improved by the combination with control nodes.

In this paper, we extend our previous research on GNP-RL and propose an algorithm that integrates the GNP-RL and control nodes in order to create an efficient portfolio optimization model for given multi-brands. The features of the proposed method compared with other traditional methods are as follows: The GNPcn method makes a stock trading strategy considering the recommendable information of technical indices and candlestick charts (Izumi, Yamaguchi, Mabu, Hirasawa, & Hu, 2006) for efficient trading decision making. Reinforcement Learning is also used in this paper for taking appropriate actions.

Section 2 presents the literature review. Section 3 describes the proposed GNPcn approach to be studied in this paper. In Section 4, we explain the optimization algorithm in brief. Section 5 presents experimental environments, conditions and results using GNPcn method. The trading profits are presented and compared with both the traditional GNP method and Buy&Hold method. Finally, Section 6 concludes this paper.

2. Literature review

In recent decades, the portfolio problem in financial engineering has received a lot of attention. The classical portfolio problem can be described like the following: Given a finite amount of resources, and the process to which these resources can be allocated, we need to get the allocation which maximizes a given goal function and minimizes the risk simultaneously. The foundation of portfolio optimization was laid by Markowitz (1959), where he proposed a mean-variance optimization model for designing an optimum portfolio based on the idea of minimizing risk and maximizing expected returns. As we know, it is difficult to acquire good estimates for the expected returns, and the calculation of risk becomes more and more complex as the number of available assets grow. In the case of linear constraints, the problem can be solved efficiently by parametric quadratic programming. However, there are many real-world non-linear constraints which limit the number of different assets in a portfolio. As a consequence, evolutionary computation was developed to calculate the optimal portfolio since portfolio problems make the search space become larger. In this paper, how to allocate the given money to a certain number of fixed brands is discussed, which is different from the classical mean-variance optimization model.

Over the last few decades, various approaches have been applied to several financial problems, especially for stock market activities. Generally speaking, these approaches can be separated into two categories: statistical and AI. The statistical methods are widely used to predict the stocks based on the past time-series data. The traditional statistical approaches include ARMA method (Box & Jenkins, 1976), the threshold Autoregressive model (Tong & Lim, 1980), Smooth Transition Autoregressive model (STAR) (Sarantis, 2001), the Autoregressive Conditional Heteroscedastic

(ARCH) (Engle, 1982) and multiple Linear Regression model. These methods rely on assumption of linearity among variables and normal distribution. However, with statistical models, problems arise when the variance in the time series increase or when non-linear processes exist in the time series. On the other hand, AI approaches, with the increasing need for more efficient trading models in the stock market, has been confirmed to outperform the conventional statistical models for that it overcomes the limitation of such an assumption (Enke & Thawornwong, 2005).

As a main approach in the AI field, Artificial Neural Networks (ANNs) has been widely used for its ability to forecast financial performance. Dropsy (1996) uses ANNs as a non-linear prediction tool to forecast international equity risk, in which both linear and non-linear forecasting results outperform the random work. Lam (2004) applied the back-propagation algorithm to integrate fundamental and technical analysis for financial performance prediction. The experimental results show that the ANNs outperform the benchmark. Both the forecasting and decision models are significantly outperforming the benchmark market performance. However, despite the wide spread use of ANNs in the financial domain, there are significant problems to be addressed. Since ANNs are data-driven model, the underlying rules in the data are not always apparent, which leads to so-called black-box models, and investors cannot benefit from the knowledge discovery in the analytic process. In addition, the buried noise and complex dimensionality of the stock market data make it difficult to learn or re-estimate the ANNs parameters (Kim & Han, 2000).

Genetic Algorithms (GA), as one of the most popular heuristic optimization techniques, were originally developed by Holland (1975). Subsequently, GA had been applied to many optimization problems in engineering and operations research. GA is a good tool for optimization problems since they make no restrictive assumptions about the solution space. Lin considered the multi-objective genetic algorithm for portfolio selection problem (Lin, Wang, & Yan, 2001). Oh proposed a new portfolio selection algorithm based on portfolio beta by using genetic algorithm (Oh, Kim, Min, & Lee, 2006). Moreover, Lin and Liu (2008) considered Markowitz's model with minimum transaction lots and they presented three other models using GA as their solver. Deck used GA to train a neural network trading system. However, when GA was applied to the portfolio optimization, the problem is that many chromosomes are coded into the same portfolio, or similar chromosomes may be coded into very different portfolios which makes it more difficult for GA to produce better chromosomes from good ones. These problems multiply the GA's search space and makes GA less efficient in finding the optimal portfolio.

Genetic programming (GP), which has been described by Koza (1992), can be considered as an extension of GA. It uses tree-like individuals that can represent mathematical expressions. So far, GP has been applied to wide range financial fields such as stock trading system (Potvin, Soriano, & Vallee, 2004), bankruptcy prediction (Etemadi, Rostamy, & Dehkordi, in press), and etc. In comparison with GA, GP allows the optimization of much more complicated structures and can therefore be applied to a greater diversity of problems (Sette & Boullart, 2001). Recent applications of GP has been done, when the notion of risk is not considered (Marney, Fyfe, Tarbert, & Miller, 2001). Although GP are widely used in the financial field, it occasionally causes some bloating problems for its tree structure.

As another tool based on expert knowledge, fuzzy logic can be used for stock market forecasting domain either independently or hybridized with other methods. For instance, Romahi and Shen (2000) developed an evolving rule based expert system to forecast the financial market activity, where the fuzzy logic and rule induction were combined together to obtain a promising method. However, fuzzy logic still has its limitation when applied to the stock

market. That is, when designing the fuzzy models for forecasting or trading, we need to get the expert knowledge prior to it.

Though there are many AI approaches have been applied to the financial field, due to such kinds of bottlenecks, we proposed GNP and GNP-RL as an extended method of GA and GP and its effectiveness has been confirmed in our former studies. In this paper, GNP with control node (GNPcn) has been firstly applied to the multi-brands optimization model for the stock trading.

3. Genetic Network Programming with control nodes

In this section, Genetic Network Programming (GNP) with control node is explained briefly. Basically, GNP is an extension of GP in terms of gene structures. The original idea is based on the more general representation ability of directed graphs than that of trees. The graph structure of GNP has some inherent characteristics such as compact structures and an implicit memory function that contribute to creating effective action rules.

3.1. Basic structure of Genetic Network Programming

The traditional GNP is composed of a start node, judgment nodes and processing nodes, which are connected to each other. Fig. 1 shows a basic structure of GNP. Judgment nodes have if-then type branch decision functions, which return judgment results for assigned inputs and determine the next node. Processing nodes take buying and selling actions for the stock trading. While judgment nodes have conditional branches, processing nodes have no conditional branches. Start node is used to determine the first node to be executed, which has no functions and no conditional branches. Once GNP is booted up, the execution starts from the start node, then the next node is determined according to the connection from the current activated node. If the current node is the judgment node, the next node is determined by the judgment result as well as its connection. When the current node is the processing node, the next node is only determined by the connection from the processing node. GNP has two kinds of time delays: one kind of time delays spend on judgment or processing,

and another spend on node transition. In this paper, the role of the time delays is to determine the maximum number of technical indices and candlestick information to be considered when GNP determines buying or selling at a certain day. In conventional GNP, the current node is not compulsorily transferred to the start node and there is no terminal node in GNP. Therefore, once GNP is booted up, the successive activation of GNP system is carried out according to the network flow until the time limit.

The genotype expression of GNP node is also shown in Fig. 1. Fig. 1 describes the gene of node i , then the set of these genes represents the genotype of GNP individuals. Concretely speaking, K_i represents the node type, $K_i = 0$ means start node, $K_i = 1$ means judgment node and $K_i = 2$ means processing node. ID_i represents an identification number of the node function, e.g., $K_i = 1$ and $ID_i = 2$ mean the node is J_2 . Q_{ip} means Q value which is assigned to each state and action pair. In this method, “state” means the current node, and “action” means the selection of a sub-node (node function). $C_{ip}^A, C_{ip}^B, \dots$ show the node number of the next node. Judgment node determines the upper suffix of the connection genes to refer to depending on the judgment result. If the judgment result is “B”, GNP refers to C_{ip}^B and d_{ip}^B . Processing nodes always refer to C_{ip}^A and d_{ip}^A because they have no conditional branch. d_{ip} ($1 \leq p \leq m_i, m_i$ is the number of sub-nodes in judgment and processing nodes) is the time delay spent on the judgment or processing at node i , while $d_{ip}^A, d_{ip}^B, \dots$ are time delays spent on the node transition from node i to the next node.

Like other evolutionary algorithms, crossover, mutation and selection are used as the genetic operators of GNP. The outline of evolution is described as follows:

- Initialize the first population and calculate the fitness of the population;
- Generate new individuals for the next generation by tournament selection and genetic operations.
- Calculate the fitness of the new individuals.
- Repeat 2–3 until the terminal condition meets.

3.2. GNP with control nodes (GNPcn)

In conventional GNP, since the current node is not compulsorily transferred to the start node, there is a possibility that some of the nodes are not used. Therefore, in this paper, GNP with control nodes (GNPcn) is used (see Fig. 2) to solve the multi-brands problem. GNPcn starts from a control node, and the current node is transferred back to one of the control nodes after executing a

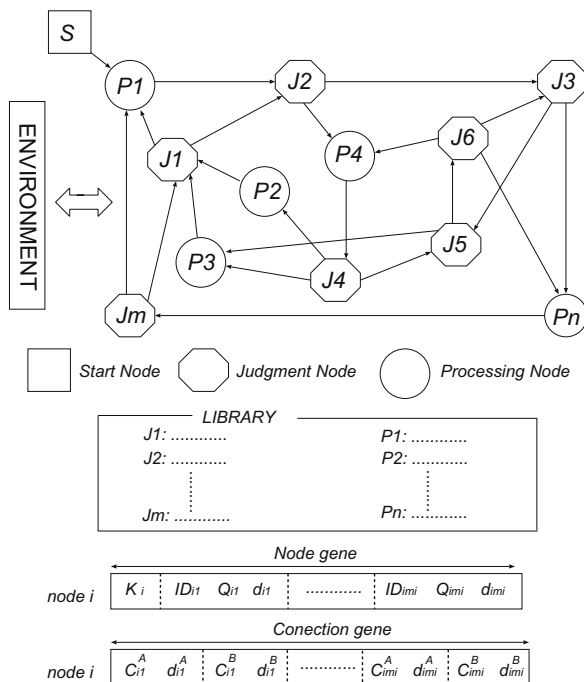


Fig. 1. The basic structure of GNP.

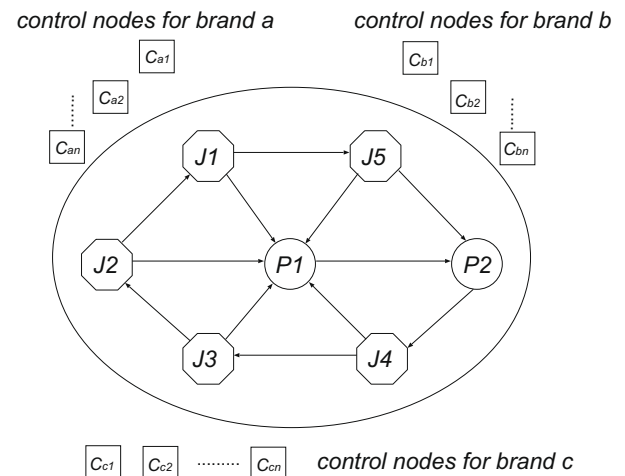


Fig. 2. The basic structure of GNPcn.

certain number of processing nodes. Consequently, the performance of GNPcn improves because the increase of the number of control nodes contributes to searching a solution space widely and finding many distinguished trading rules. Especially, the number of the control nodes can be considered as breadth of the search of GNP, while the number of processing nodes activated per control node can be considered as depth of the search of GNP.

In addition, the design on what kinds of branches the judgment nodes should have is so important similarly to the design of the processing nodes in GNP. In this case, the transition or the behavior of GNP is mainly determined by which branch the judgment node selects. Therefore, GNP should have the possibility for selecting many judgment results in GNP. In order to overcome such a problem as limiting the usage of nodes, in this paper, GNPcn is proposed and basic studies are done where the number of control nodes is fixed and the number of processing nodes activated per control node is evolved.

Fig. 2 shows the basic structure of GNPcn. GNPcn has several control nodes, judgment nodes and processing nodes. GNPcn uses one of the groups of control nodes for one brand of stocks, so that GNPcn could deal with multi-brands. When GNPcn deals with brand “a”, GNPcn starts its node transition from control node “ C_{a1} ”, and the current activated node returns to one of the control nodes ($C_{a1}, \dots, C_{an}$) successively after transiting m processing nodes from the last control node. This procedure is as follows:

- First, the activated node is determined by the control node of each brand.
- Execute the judgment or processing function of the current activated node of the brand, then determines the next node which is executed in the next time period.
- The next node is determined by the next control node (such as $C_{a1}, \dots, C_{an}$) after m processing nodes are executed from the last control node.

3.3. Genetic operations of GNPcn

Fig. 3 shows the whole flowchart of GNPcn. In this subsection, the genetic operators in the evolution phase are introduced. GNPcn with RL has three kinds of genetic operators: selection, crossover and mutation. In GNPcn, mutation operation could be executed not only on the connections between nodes but also on the node functions and the number of processing nodes activated per control node.

3.3.1. Crossover

Crossover is executed between two parents and generates two offspring. The procedure of crossover is as follows.

- (1) Select two individuals using tournament selection twice and produce them as parents.
- (2) Each node is selected as a crossover node with the probability of P_c .
- (3) Two parents exchange the genes of the corresponding crossover nodes.
- (4) Generated new individuals become the new ones of the next generation.

If GNP exchanges the genes of judgment nodes, it must exchange all the genes with suffix A, B, C, ... simultaneously.

3.3.2. Mutation

Mutation is executed in one individual and a new one is generated. The procedure of mutation is as follows.

- (1) Select one individual as a parent using tournament selection.
- (2) Mutation operation.

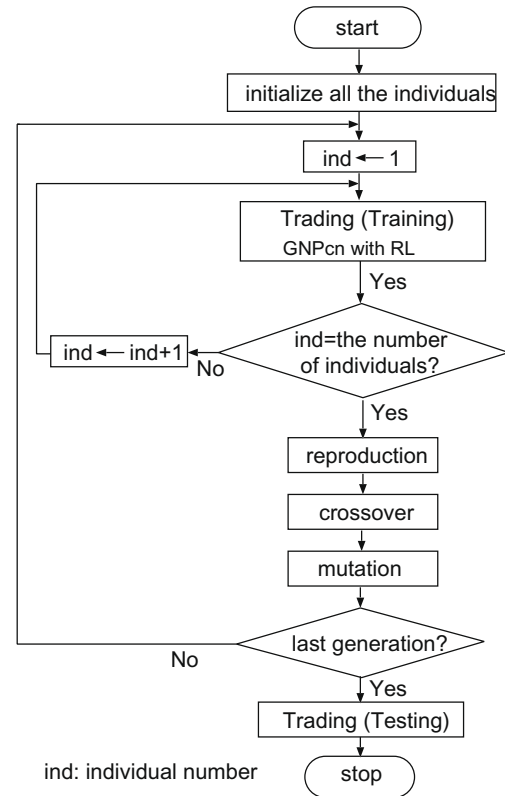


Fig. 3. Flowchart of GNPcn.

- Change connection: Each node branch ($C_{ip}^A, C_{ip}^B, \dots$) is selected with the probability of P_m , and the selected branch is reconnected to another node.
- Change node function: Each node function (ID_{ip}) is selected with the probability of P_m , and the selected function is changed to another one.
- Change node number: The number of processing nodes activated per control node (m) is changed to the other value with the probability of P_m .

(3) Generated new individual becomes the new one of the next generation.

3.4. Learning phase of GNPcn

In this paper, we use GNPcn with Sarsa learning which is one kind of reinforcement learning methods. Sarsa can obtain Q values which estimate the sum of the discounted rewards obtained in the future. Suppose an agent selects an action a_t at state s_t at time t , a reward r_t is obtained and an action a_{t+1} is taken at the next state s_{t+1} . Then $Q(s_t, a_t)$ is updated as follows:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

α is a step size parameter, and γ is a discount rate. The state means the current node and action means the selection of the sub-node in this paper.

4. Portfolio optimization algorithm using GNPcn

4.1. Technical indices and candlestick chart

In our proposed portfolio optimization model, technical indices and candlestick chart are used as judgment functions. Concretely

speaking, each judgment node uses one of the following technical indices for its judgment: Rate of Deviation from moving average (ROD), Relative Strength Index (RSI), Rate of Change (ROC), Volume Ratio, Rank Correlation Index (RCI), Stochastics, Golden/Dead cross and Moving Average Convergence and Divergence (MACD).

As a judgment function, a candlestick chart was also used in order to elicit technical knowledge. The candlestick chart is a useful tool to visualize the stock prices so that investors can detect patterns which can be used to predict future stock price movements. As illustrated in Fig. 4, the candlestick chart consists of a rectangle and two shadow lines. The rectangle which is called the real body indicates the difference between the opening value and closing value of the stock. If the real body of the candlestick chart shows that the opening value is higher than the closing value, the candlestick chart is called black candlestick. On the other hand, if the closing value is higher than the opening value, the candlestick chart is called white candlestick. The white candlestick implies the rising signal of the stock price and the black candlestick implies the falling signal. The stock price patterns which are represented by the candlestick chart shapes give important clues to predict future stock price movements. Thus, the technical knowledge from the candlestick chart was used as a key information for predicting stock price movements in this study.

4.2. Judgment function of GNPcn

Table 1 shows the node functions of GNPcn. When a current activated node i is a judgment node, first, one Q value is selected from $Q_{i1}, \dots, Q_{im_i}$ based on ϵ -greedy policy (m_i is the number of sub-nodes in judgment and processing nodes). Then, the corresponding function (ID_{ip}) is selected. The gene ID_{ip} shows a technical index or a candlestick GNP judges at node i .

Each technical index has its own judgment function according to the technical value as shown in Fig. 5, where x-axis shows the value of each technical index, and the y-axis shows the IMX output (IMX is the Importance Index which has been proposed in our former research (Mabu et al., 2007)) of the judgment function. The shape of IMX functions is fixed in our study according to the knowledge from the real stock market. A, B, C, ... correspond to judgment results. Suppose Q_{i1} and the corresponding $ID_{i1} = 1$ are selected (judgment of rate of deviation), and if the rate is more than 0.1, the judgment result becomes E, and the next node number becomes C_{i1}^E .

As shown in Fig. 4, the candlestick chart is used as one of the judgment functions in this paper. The proposed method has judg-

Table 1

Functions of control nodes, judgment nodes and processing nodes.

0	Control node
1	Judgment node to check the technical indices
1	Judgment node to check the candlestick charts
2	Processing node for buying action
2	Processing node for selling action

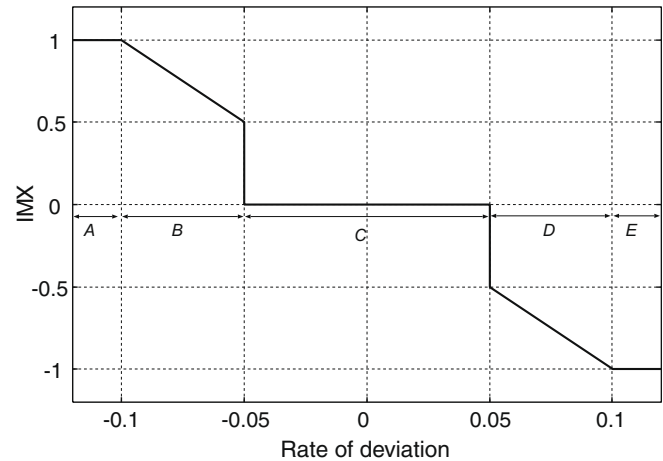


Fig. 5. IMX functions in judgment nodes (in case of ROD).

ment nodes which check candlestick chart patterns. The judgment function of the candlestick chart is executed as follows. When the selected sub-node has a judgment function of the candlestick chart, GNP judges the current day's candlestick by the price information. In our study, we use nine typical candlestick chart patterns as shown in Fig. 4, and following is a brief introduction about these patterns:

- Long white candlestick: It indicates the signal of uptrend movement (the longer the body, the more significant the price increase).
- Long black candlestick: It indicates the signal of downtrend movement (the longer the body, the more significant the price decrease).

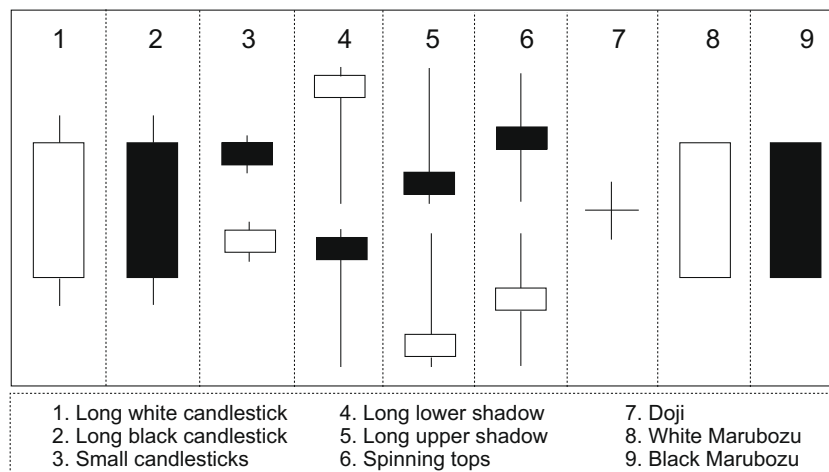


Fig. 4. Candlestick chart patterns.

- Small candlesticks: It indicates that neither bullish nor bearish signal could control the movement and prices finished about where they started.
- Long lower shadow: It indicates bullish signal (the longer the lower wick, the more reliable the signal).
- Long upper shadow: It indicates bearish signal (the longer the upper wick, the more reliable the signal).
- Spinning tops: It indicates a standoff of bullish and bearish signal, which are neutral patterns.
- Doji: It indicates a standoff, and a turning point could be developing.
- White Marubozu: It indicates dominant bullish trades with continued bullish trend.
- Black Marubozu: It indicates dominant bearish trades with continued bearish trend.

As an example, when the candlestick pattern is “3”, GNPCn selects third branch to transfer to the next node.

4.3. Processing function of GNPCn

When the current node is a processing node, the following will be carried out:

- (1) First, one Q value is selected from $Q_{i1}, \dots, Q_{imi}$ based on ϵ -greedy policy. That is, a maximum Q value among $Q_{i1}, \dots, Q_{imi}$ is selected with the probability of $1 - \epsilon$, or a random one is selected with the probability of ϵ . Then, the corresponding ID_{ip} is selected.
- (2) Determine buying or selling:
 - In the case of $ID_{ip} = 0$ (buy): GNPCn buys the stock of brand $b \in B$ as much as possible using $Initial(V, b)$, where $Initial(V, b)$ is the initial budget of brand b used for validation. It is explained in the next subsection.
 - In the case of $ID_{ip} = 1$ (sell): GNP sells all of the stocks of brand $b \in B$ in hand.
- (3) If ID_{ip} is selected, the next node number becomes C_{ip}^A when the current node is transferred to the next node.

4.4. Multi-brands optimization algorithm

In this section, how to determine the initial budget of each brand is described in the dealing of stocks using GNP with control nodes. This portfolio optimization system has been constructed by training phase and validation phase (see Fig. 6). Especially, the portfolio does not change during the validation period, although it changes generation by generation in the training period.

4.4.1. Notations

- $Initial(t)$: initial budget for training.
- $Initial(v)$: initial budget for validation.
- $Initial(t, b, n)$: initial budget of brand b in the n th generation for training.
- $Initial(v, b)$: initial budget of brand b for validation.
- T : temperature parameter.
- B : set of brands.
- b : brand index.
- N : number of generations.
- n : generation number.
- $Sell(t, b, n)$: sum of the money obtained by selling the stocks of brand b during the dealing in the n th generation for training.
- $Buy(t, b, n)$: sum of the money paid by buying the stocks of brand b during the dealing in the n th generation for training.

- $Profit(t, b, n)$: money gained or lost during the dealing of brand b in the n th generation for training, where $Profit(t, b, n) = Sell(t, b, n) - Buy(t, b, n)$.
- $Profitability(t, b, n)$: profitability during the dealing of brand b in the n th generation for training, where $Profitability(t, b, n) = Profit(t, b, n) / Initial(t, b, n)$.
- $Profit(v, b, d)$: money gained or lost during the dealing of brand b until day d for validation.

We use GNPCn for determining the portfolio of the stock brands, where each brand corresponds to each group of control nodes like shown in Fig. 2. In GNPCn, judgment nodes check the technical indices and candlestick chart patterns, and processing nodes work for buying or selling stocks.

One of the groups of control nodes is assigned to each brand $b \in B$. The current activated node returns to one of the control nodes of each brand after transiting m processing nodes from the last control node. That is, the current activated node returns to C_{b2} firstly, and next $C_{b3}, C_{b4}, \dots, C_{bn}$. Trading timing is determined by the GNPCn evolution.

The fitness function of the GNPCn in the n th generation is defined as follows,

$$Fitness(n) = \sum_{b \in B} Profit(t, b, n)$$

In the following, how to determine the portfolio is explained.

4.4.2. Training phase

The initial budget $Initial(t, b, 1)$ of brand b in the first generation is calculated by

$$Initial(t, b, 1) = \frac{Initial(t)}{|B|} \quad (1)$$

After the first generation, we can get the following profit and profitability

$$Profit(t, b, 1) = Sell(t, b, 1) - Buy(t, b, 1)$$

$$Profitability(t, b, 1) = \frac{Profit(t, b, 1)}{Initial(t, b, 1)}$$

Then, the initial budget $Initial(t, b, 2)$ of the brand b in the second generation can be calculated by Eq. (2) considering the profitability gained in the first generation

$$Initial(t, b, 2) = \frac{\exp(Profitability(t, b, 1)/T)}{\sum_{b \in B} \exp(Profitability(t, b, 1)/T)} Initial(t) \quad (2)$$

Likewise, the initial budget $Initial(t, b, n+1)$ of the brand b in the $(n+1)$ th generation can be calculated by Eq. (3) considering the profitability gained in the n th generation

$$Initial(t, b, n+1) = \frac{\exp(Profitability(t, b, n)/T)}{\sum_{b \in B} \exp(Profitability(t, b, n)/T)} Initial(t), \quad (3)$$

where

$$Profit(t, b, n) = Sell(t, b, n) - Buy(t, b, n)$$

$$Profitability(t, b, n) = \frac{Profit(t, b, n)}{Initial(t, b, n)}$$

4.4.3. Validation phase

The initial budget $Initial(v, b)$ of brand b in the validation phase is given as follows.

$$Initial(v, b) = \frac{\exp(Profitability(t, b, N)/T)}{\sum_{b \in B} \exp(Profitability(t, b, N)/T)} Initial(v) \quad (4)$$

where $Initial(v)$ is the initial budget for the validation.

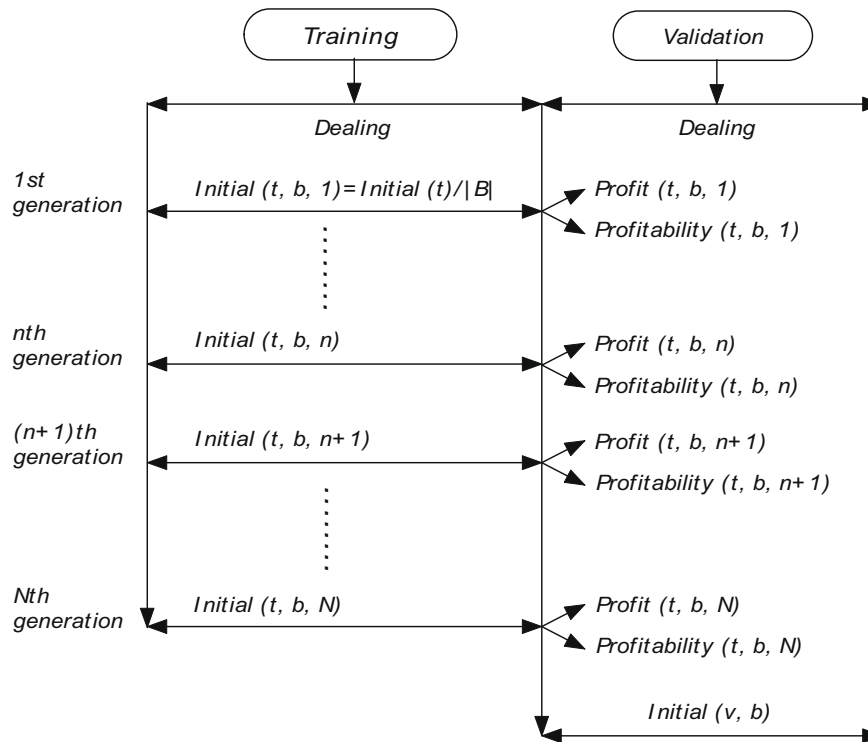


Fig. 6. Training and validation phase.

GNPcn individual starts its operation from one of the control nodes in the validation and the activated node is transferred to a judgment node or a processing node. At the processing node, the trading is executed using the opening price of the day. The concrete procedure of the trading is as follows. Do the following for each brand until the end of the trading:

- If the current node is a judgment node, it determines the next node depending on the judgment result. If the current node is a processing node, it transits to the next node after buying or selling stocks.
- Calculate the available budget of each brand whenever the buying signal occurs in the processing nodes during the transition of each brand.
- When the processing node is executed m times from the last control node, the next node is determined by the next control node.
- Calculate the profit and profitability of each brand at the end of the trading.

5. Simulations

To confirm the effectiveness of GNPcn for the portfolio optimization system, we carried out the trading simulations using 10 brands selected from the companies listed in the first section of Tokyo stock market in Japan (see Table 2). The simulation period is divided into two periods: one is used for training and the other is used for testing.

- Training: January 4, 2001–December 30, 2003 (737 days).
- Testing: January 5, 2004–December 30, 2004 (246 days).

We suppose that the initial funds, i.e., $Initial(t)$ and $Initial(v)$ are 50,000,000 Japanese yen in both periods.

Especially, there is an advantage of GNPcn over the conventional GNP: In the simulations, GNPcn is compared with the tradi-

tional GNP in which one GNP individual deals with only one brand, so we evolved 10 different GNP populations. When we use GNPcn, one GNPcn has 10 groups of control nodes, each of which deals with one brand, so one GNPcn can deal with 10 brands.

5.1. Settings of GNPcn

Table 3 shows the parameters of the evolution of GNPcn. GNPcn uses judgment nodes which judge the technical indices and candlestick charts. The technical indices are calculated using three kinds of calculation periods (short-term, medium-term and long-term) except Golden/Dead cross and MACD. Therefore, the number of kinds of judgment nodes is 21 (including one candlestick judgment). The number of processing functions is two: buying and selling. Five control nodes are assigned to each brand and we tested 10 brands, i.e., the total number of control nodes is 50. The total number of nodes in each individual is 80 including 10 processing nodes, 20 judgment nodes and 50 control nodes. However, the functions ID_{ip} in sub-nodes are determined randomly at the beginning of the first generation, and changed appropriately by evolution.

The initial connections between nodes are also determined randomly at the first generation. At the end of each generation, 179 new individuals are produced by mutation, 120 new individuals are produced by crossover, and the best individual is preserved. The other parameters are the ones showing good results in the simulations. The initial Q values are set at zero.

5.2. Simulation results

In Table 4, we test the various values of T in Eq. (2), which is called temperature parameter. As we know, according to the Boltzmann distribution theory, different T value defines different strength of distribution. When it is applied to this portfolio study, T value indicates the initial budget of each brand distributed by GNPcn. If $T \rightarrow \infty$, GNPcn distributes the initial budget to each stock

Table 2

Profits in the testing simulations {Profit[yen](profitability[%])}.

Brand	GNPcn	GNP-RL	GNP-candlestick	GA	Buy&Hold
NEC Corporation	–	43,400(0.9)	–126,150(–2.5)	–531,000(–10.6)	–1,026,000(–20.5)
Fuji Heavy Ind.	–	188,800(3.8)	97,700(2.0)	173,000(3.5)	–189,000(–3.8)
East Japan Railway	–	418,800(8.4)	96,550(1.9)	7,590(0.2)	477,000(9.5)
KDDI	–	316,233(6.3)	–76,600(–1.5)	–273,000(–5.5)	–576,000(–11.5)
Nomura Holdings, Inc.	–	638,933(12.8)	–293,785(–5.9)	–374,087(–7.5)	–985,500(–19.7)
Shin-Etsu Chemical	–	133,800(2.7)	7,250(0.1)	–539,400(–10.8)	–264,000(–5.3)
Sony	–	202,766(4.1)	280,500(5.6)	112,000(2.2)	150,000(3.0)
Tokyo Electric Power	–	116,100(2.3)	–65,500(–1.3)	360,750(7.2)	262,500(5.3)
Hitachi	–	403,366(8.1)	472,300(9.4)	276,300(5.5)	336,000(6.7)
Nissan	–	464,966(9.3)	590,750(11.8)	372,000(7.4)	450,000(9.0)
Average	4,262,714(8.5)	292,716 (5.9)	98,302(2.0)	–41,585 (–0.8)	–136,500 (–2.7)

Table 3

Parameter conditions for evolving GNP.

Number of individuals = 300 (mutation:179, crossover:120, elite:1)
Number of nodes = 80 (judgement node = 20, Processing node = 10, control node = 50)
Number of sub-node in each node = 2
$P_c = 0.1$, $P_m = 0.03$, $\alpha = 0.1$, $\gamma = 0.3$, $\epsilon = 0.1$

Table 4Average fitness values [million yen] in training period with different T .

T	0.001	0.003	0.005	0.01	0.03	0.05
Fitness	133.5	143.2	136.2	125.6	110.1	108.2

evenly. On the other hand, if $T \rightarrow 0$, all the money is distributed to the stock brand which obtain the best profitability.

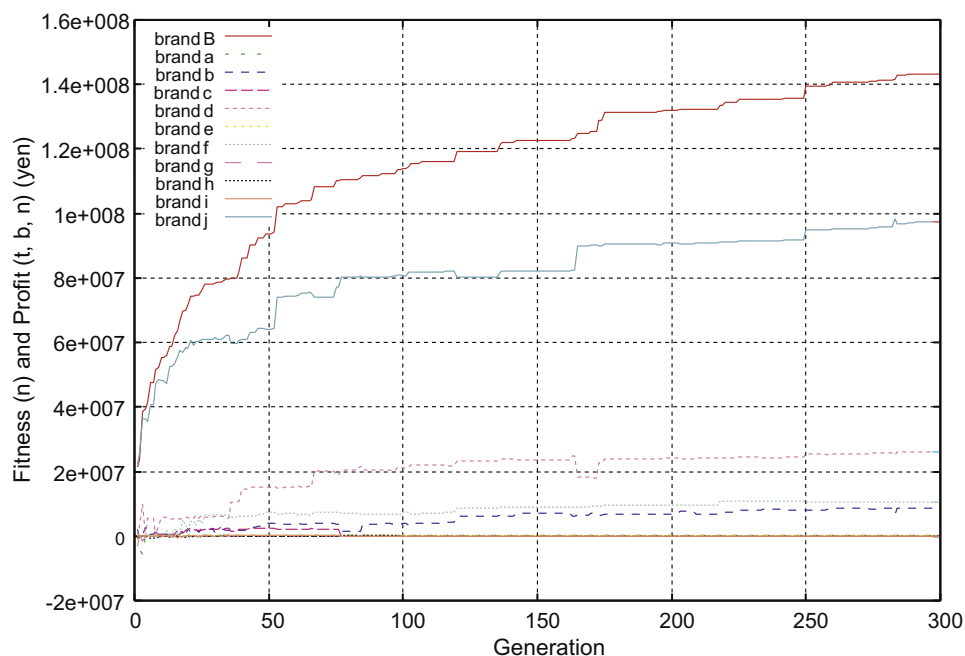
Table 4 shows the average fitness values over 30 independent simulations in the training when the parameter T is set to different values. From the results, it is clarified that the we can get a good fitness value in the training period when T is set to 0.003. There-

fore, we set the value of 0.003 for the temperature parameter T in our simulations.

Fig. 7 shows the fitness and profit curves, i.e., $Fitness(n)$ and $Profit(t, b, n)$ of each brand $b \in B$ in the n th generation using the data from 2001 to 2003, and the line is the average value over 30 independent simulations. From Fig. 7, we can see that GNPcn with reinforcement learning can obtain larger profits for the training data as the generation goes on.

Fig. 8 shows the initial budget change of the 10 brands, i.e., $Initial(t, b, n)$ in the training period, while Fig. 9 shows the profitability change of each brand, i.e., $Profitability(t, b, n)$ in the training period. From these figures, it is clear that the brand which can obtain larger profitability can also get the initial budget more than other brands. The optimization of the proposed portfolio model has been shown consequently.

Table 2 shows the profit and profitability in the testing term using the data in 2004. The values in Table 2 are the average profit and profitability of 30 independent simulations with different random seeds. For the comparison, Table 2 also shows the results of GNP-RL, GNP-candlestick (Izumi et al., 2006), GA and Buy&Hold method which is often considered as a benchmark in trading stocks simulations. Buy&Hold buys as much stocks as possible at the opening of the market on the first day in the simulations, and sells

**Fig. 7.** Fitness and profit curves of 10 brands in the training period.

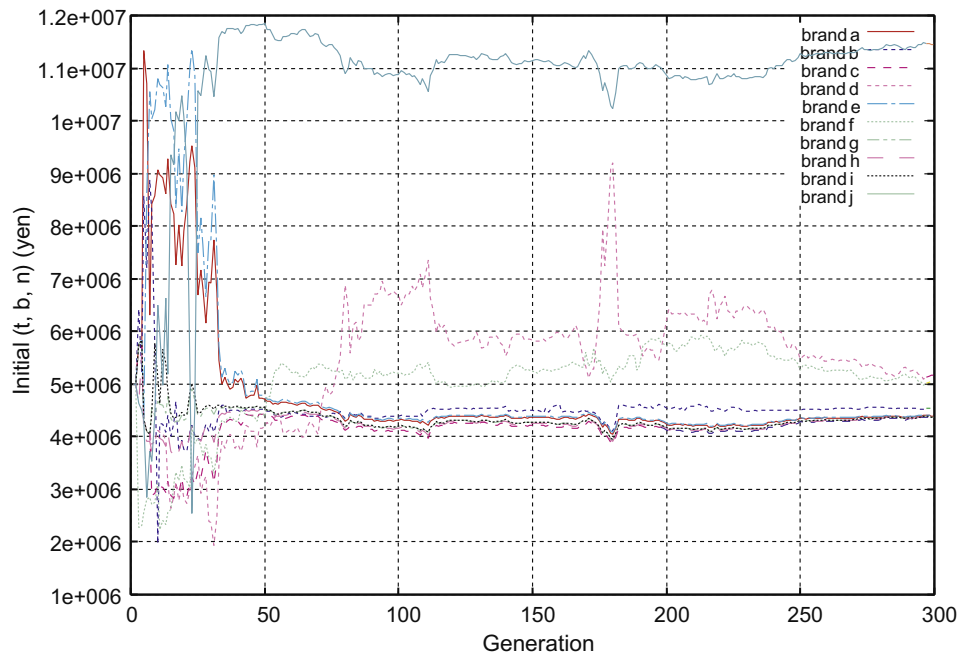


Fig. 8. Initial budget change of 10 brands in the training period.

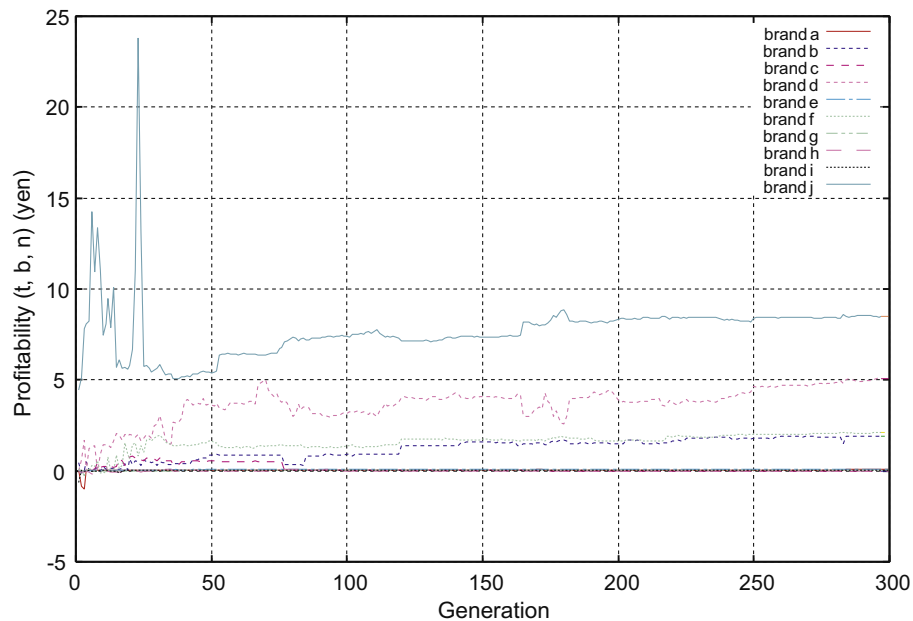


Fig. 9. Profitability change of 10 brands in the training period.

all the stocks at the opening on the last day. From Table 2, the proposed method can obtain larger profits than Buy&Hold and all the other traditional methods in most of the trade in 10 brands. Since the initial budget of the traditional methods and Buy&Hold are 5,000,000 yen respectively, we set the initial budget of the proposed model at 50,000,000 yen. This is because GNPcn deals with all of the 10 brands simultaneously, while in the case of conventional GNP method, one GNP deals with only one brand. In order to compare these methods under the same condition, we set the initial budget of GNPcn at 10 times of other methods. As a result, the average profitability can be calculated and compared with each other. Especially, the profit for each brand can be calculated in the

traditional methods, however, in the proposed model, GNPcn deals with all of the brands, so only the total profit is described.

Fig. 10 shows the $Profit(v, b, d)$ during the dealing of brand $b \in B$ until day d using the data of 2004. From Fig. 10, we can see that the profit keep increasing during the testing period. The advantage of the proposed method is to determine the distribution of the initial budget to each brand automatically, and the brands which obtain larger profitability can have the initial budget more than other brands. As a result, by this efficient portfolio optimization system, GNPcn could obtain much profit in the trading of those brands.

Fig. 11 shows the average ratio of the nodes used in the test period over 30 independent simulations in order to see which nodes

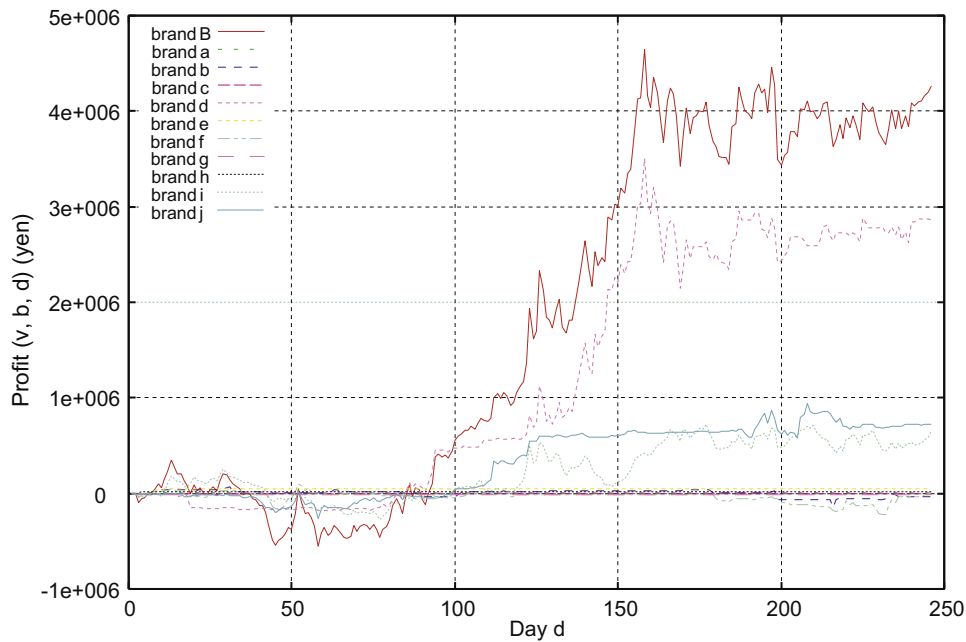


Fig. 10. Profits change of 10 brands in the testing period.

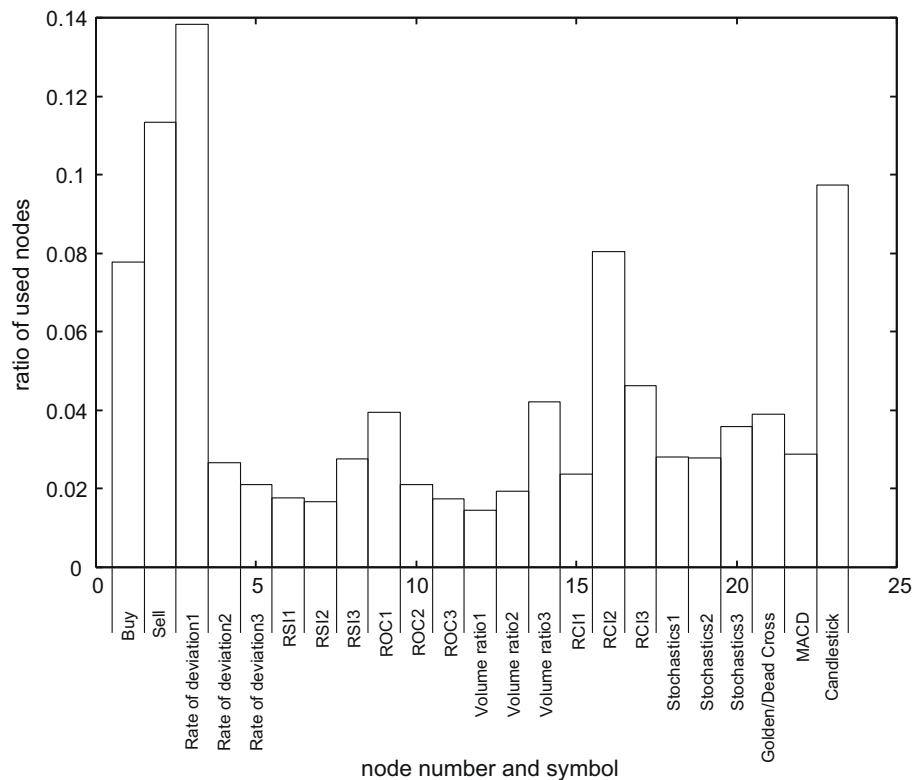


Fig. 11. Ratio of nodes used by GNPcn in the testing period.

are used and which are most efficient for stock trading model. The total number of node functions is 23, while each processing node has a node number of (1–2), and each judgment node has a node number of (3–23). The x-axis shows the kinds of nodes, while the y-axis shows the average ratio of the used nodes. From the figure, we can see that the processing nodes are used to determine buying and selling stocks, and the judgment nodes of “Rate of Deviation 1”

and “candlestick” are frequently used. Thus it can be said that GNPcn judges that these node functions are important to determine stock trading. GNPcn can automatically determine which nodes should be used in the current situation by evolving node functions and connections between nodes, in other words, GNPcn can optimize the combination of technical indices and candlestick charts used for this multi-brands stock trading model.

6. Conclusions

In this paper, we proposed a multi-brands optimization algorithm by using GNPCn to check the information of technical indices and candlestick chart patterns. Compared to conventional GNP, GNPCn has many control nodes, and each group of control nodes is assigned to each stock brand for creating effective portfolio strategy. Since GNPCn can adjust the distribution of the initial budget to each brand at each generation and more budget is assigned to the brands with larger profitability, we can create the effective portfolio optimization system and get more profit. We carried out the simulation of GNPCn method using stock data of 10 brands for 4 years. In the simulations, GNPCn is compared with several conventional methods and Buy&Hold. From the results, it is clarified that GNPCn performs much better in terms of the profits than traditional GNP methods, GA and Buy&Hold method. It shows the efficiency of the proposed GNPCn method for dealing with the portfolio optimization problem.

There remain some further study in the future. First, the algorithm presented can be further improved by modifying evolutionary operators, especially mutations. The fitness function can also be improved to increase the efficiency of the method. Moreover, additional efforts should be spent on the methods of portfolio validation in order to eliminate unacceptable solutions at the moment of its creation. We will also evaluate the proposed method comparing with other methods in the financial market.

References

- Box, G. E. P., & Jenkins, G. M. (1976). *Time series analysis: forecasting and control*. San Francisco, CA: Holden-Day.
- Dempster, M. A. H., & Jones, C. M. (2001). A real-time adaptive trading system using genetic programming. *Quantitative Finance*, 1, 397–413.
- Dropsy, V. (1996). Do macroeconomic factors help in predicting international equity risk premia? *Journal of Applied Business Research*, 12, 120–132.
- Eguchi, T., Hirasawa, K., Hu, J., & Ota, N. (2006). Study of evolutionary multiagent models based on symbiosis. *IEEE Transactions on Systems, Man and Cybernetics B*, 36(1), 179–193.
- Engle, R. F. (1982). Autoregressive conditional heteroskedasticity with estimates of the variance of UK inflation. *Econometrica*, 50, 987–1008.
- Enke, D., & Thawornwong, S. (2005). The use of data mining and neural networks for forecasting stock market returns. *Expert Systems with Applications*, 29, 927–940.
- Etemadi, H., Rostamy, A. A., Dehkordi, H. F. (in press). A genetic programming model for bankruptcy prediction: Empirical evidence from Iran. *Expert Systems with Applications*, 36(2P2), 3199–3207.
- Eto, S., Mabu, S., Hirasawa, K., Hu, J. (2006). Genetic network programming considering the evolution of breadth and depth. In *Proceedings of SICE-ICASE international joint conference* (pp. 5504–5508).
- Goldberg, D. E. (1989). *Genetic Algorithm in search, optimization and machine learning*. Addison-Wesley.
- Holland, J. H. (1975). *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control and artificial intelligence*. University of Michigan Press.
- Izumi, Y., Yamaguchi, T., Mabu, S., Hirasawa, K., & Hu, J. (2006). Trading rules on the stock markets using genetic network programming with candlestick chart. In *Proceedings of IEEE congress on evolutionary computation* (pp. 8531–8536).
- Kim, K.-J., & Han, I. (2000). Genetic algorithms approach to feature discretization in artificial neural networks for the prediction of stock price index. *Expert Systems with Applications*, 19, 125–132.
- Koza, J. R. (1992). *Genetic Programming, on the programming of computers by means of natural selection*. Cambridge, Mass.: MIT Press.
- Lam, M. (2004). Neural network techniques for financial performance prediction: integrating fundamental and technical analysis. *Decision Support Systems*, 37, 567–581.
- Lin, D., Wang, S., & Yan, H. (2001). A multiobjective genetic algorithm for portfolio selection problem. In *Proceedings of ICOTA, Hong Kong, December 2001*.
- Lin, C. C., & Liu, Y. T. (2008). Genetic algorithms for portfolio selection problems with minimum transaction lots. *European Journal of Operational Research*, 185(1), 393–404.
- Loraschi, A., Tettamanzi, A., Tomassini, M., & Verda, P. (1995). Distributed genetic algorithms with an application to portfolio selection problems. In D. W. Pearson, N. C. Steele, & R. F. Albrecht (Eds.), *Artificial neural networks and genetic algorithms* (pp. 384–387). Wein: Springer.
- Mabu, S., Hirasawa, K., & Hu, J. (2007). A graph-based evolutionary algorithm: Genetic network programming and its extension using reinforcement learning. *Evolutionary Computation*, 15(3), 369–398.
- Mabu, S., Chen, Y., Hirasawa, K., & Hu, J. (2007). Stock Trading rules using genetic network programming with actor-critic. In *Proceedings of IEEE congress on evolutionary computation* (pp. 508–515).
- Markowitz, H. (1959). *Portfolio selection: Efficient diversification of investments*. New York: John Wiley & Sons.
- Marney, J. P., Fyfe, C., Tarbert, H., & Miller, D. (2001). Risk adjusted returns to technical trading rules: A genetic programming approach. Computing in Economics and Finance, Society for Computational Economics, Yale University, USA, June, paper No. 147.
- Oh, K. J., Kim, T. Y., Min, S. H., & Lee, H. Y. (2006). Portfolio algorithm based on portfolio beta using genetic algorithm. *Expert Systems with Applications*, 30, 527–534.
- Potvin, J., Soriano, P., & Vallee, M. (2004). Generating trading rules on the stock markets with genetic programming. *Computers and Operations Research*, 31, 1033–1047.
- Romahi, Y., & Shen, Q. (2000). Dynamic financial forecasting with automatically induced fuzzy associations. In *Proceedings of the 9th international conference on fuzzy systems* (pp. 493–498).
- Sarantis, N. (2001). Nonlinearities, cyclical behavior and predictability in stock markets: International evidence. *International Journal of Forecasting*, 17(3), 459–482.
- Sette, S., & Boullart, L. (2001). Genetic programming: principles and applications. *Engineering Applications of Artificial Intelligence*, 14(6), 727–736.
- Skolpadungket, P., Dahal, K., & Harnpornchai, N. (2007). Portfolio optimization using multi-objective genetic algorithms. In *Proceedings of IEEE congress on evolutionary computation* (pp. 516–523).
- Tong, H., & Lim, K. S. (1980). Threshold autoregressive, limit cycles and cyclical data. *Journal of the Royal Statistical Society Series B*, 42(3), 245–292.